

```

# BLOB Principe 2.0 v1 - Modele cosmogonique avec mitose simplexiale
#
# Cadre Rudolphe Mirante - cosmogonie Blob :
#   Phase 0 : un seul simplexe originel
#   Mitose : prolifération ontologique, rythme constant
#           1 événement tous les 1000 pas Blob (indépendant de N)
#   Pas d'arête de filiation (le nouveau simplexe est libre)
#   Micro-décalage spatial pour éviter le cristal parfait
#
#   Hierarchie testée :
#     Niveau 0 : simplexe (être pur)
#     Niveau 1 : arête (énergie sombre dans la cosmologie Mirante)
#     Niveau 2 : triangle (matière noire dans la cosmologie Mirante)
#     Niveau 3 : tétraèdre (matière baryonique : seuil thermodynamique)
#
#   Hypothèse centrale à tester :
#     La dispersion D et le temps physique T_n n'émergent qu'après
#     l'apparition du premier tétraèdre. Avant : phase froide. Après :
#     chaleur croissante avec densité tétraédrique.

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv[0]):
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous = {
                chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014",
            }
            found = [(name, content.count(c)) for c, name in dangerous.items() if

```

```

        if found:
            print("ERREUR caracteres corrompus :", found)
            return False
        star = chr(42)
        bad_init = star + star + "init" + star + star
        if content.count(bad_init) > 1:
            print("ERREUR : motif corrompu init")
            return False
        triple_back = chr(96) * 3
        if content.count(triple_back) > 1:
            print("ERREUR : Markdown")
            return False
        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
    except Exception as e:
        print("Note : " + str(e))
    print("=" * 70)
    return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====

import numpy as np
import time
import json
import gc
from collections import defaultdict

# =====
# Parametres canoniques Blob (inchanges depuis A1-A4)
# =====
KAPPA_0_TARGET = 8.0    # connectivite cible (auto-regulee par dynamique)
BETA = 0.30
TAU = 0.50

# Parametres specifiques Blob 2.0
P_MITOSE = 0.001        # probabilite de mitose par pas Blob (independant de N)
EPS_DECAL = 0.05         # micro-decalage spatial relatif au rayon Rips local

OUTPUT_DIR = '/content/blob_v2_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

```

```

# =====
# Classe BlobUniverse - graphe avec mitose
# =====
class BlobUniverse:
    """
    Univers Blob avec N(t) variable par mitose simplexiale.

    Structure :
    - simplexes : liste de positions 3D (taille N(t))
    - generation : pour chaque simplexe, sa generation dans la lignee
    - parent : pour chaque simplexe (sauf l originel), l indice de son parent
    - adj : adjacences (sets)
    - edge_list : liste des aretes courantes
    """

    def __init__(self, seed=0):
        self.rng = np.random.default_rng(seed)
        self.seed = seed

        # Etat initial : UN SEUL simplexe a l origine
        self.positions = [np.array([0.5, 0.5, 0.5])] # le simplexe originel
        self.generation = [0] # generation 0
        self.parent = [None] # pas de parent
        self.adj = [set()] # pas de connexions
        self.edge_list = []
        self.deg = np.array([0], dtype=np.int32)

        self.n_tri = 0
        self.n_tetra = 0
        self.S_curr = self._compute_S()

        # Compteurs
        self.n_mitoses = 0
        self.n_swaps_accepted = 0
        self.n_swaps_proposed = 0
        self.first_edge_step = None
        self.first_triangle_step = None
        self.first_tetrahedron_step = None

    @property
    def N(self):
        return len(self.positions)

    @property
    def E(self):

```

```

        return len(self.edge_list)

def _compute_D(self):
    if self.N == 0:
        return 0.0
    mu = float(self.deg.mean())
    if mu == 0:
        return 0.0
    return float(self.deg.std()) / mu

def _compute_S(self):
    D = self._compute_D()
    return np.log(1 + self.n_tri) - BETA * D

def _rips_radius(self):
    """Rayon Rips adapte a N(t)."""
    if self.N <= 1:
        return 0.5 # arbitrairement large au debut
    c_3 = 4 * np.pi / 3
    return 1.5 * (KAPPA_0_TARGET / (c_3 * self.N)) ** (1.0/3)

def _count_tetra_through_edge(self, u, v):
    """Tetraedres impliquant l arete (u,v) dans l etat courant."""
    common = self.adj[u] & self.adj[v]
    if len(common) < 2:
        return 0
    common_list = list(common)
    count = 0
    for i, w in enumerate(common_list):
        adj_w = self.adj[w]
        for x in common_list[i+1:]:
            if x in adj_w:
                count += 1
    return count

def mitose(self, current_step):
    """
    Mitose ontologique : un simplexe donne naissance a un autre.
    Choix du parent : uniformement parmi les N(t) actuels.
    Position : decalage gaussien autour du parent.
    Pas d arete de filiation.
    Le nouveau simplexe a degre 0 (libre).
    """
    if self.N == 0:
        return

    # Choisir le parent

```

```

parent_idx = int(self.rng.integers(self.N))
parent_pos = self.positions[parent_idx]
parent_gen = self.generation[parent_idx]

# Decalage spatial
r_p = self._rips_radius()
decal = self.rng.normal(0, EPS_DECAL * r_p, size=3)
new_pos = parent_pos + decal

# Ajouter le nouveau simplexe
new_idx = self.N
self.positions.append(new_pos)
self.generation.append(parent_gen + 1)
self.parent.append(parent_idx)
self.adj.append(set())
self.deg = np.append(self.deg, 0)

self.n_mitoses += 1

def step_blob(self, current_step):
    """
    Pas Blob : tentative de swap d arete.
    Si N < 2, rien a faire.
    Si E = 0, propose juste l ajout d une arete.
    Si E > 0, swap classique : retrait + ajout.
    """
    if self.N < 2:
        return False

    # Cas special : pas d arete encore -> juste proposer ajout
    if self.E == 0:
        return self._step_blob_add_only()

    # Choisir si on fait swap ou juste ajout (dependant de connectivite)
    # Si la connectivite est en-dessous de la cible, plus de proba d ajout pu
    E_target = int(KAPPA_0_TARGET * self.N / 2)
    if self.E < E_target:
        # Probabilite d ajout pur croissante quand E sous-target
        deficit_ratio = (E_target - self.E) / max(E_target, 1)
        if self.rng.random() < deficit_ratio:
            return self._step_blob_add_only()

    return self._step_blob_swap(current_step)

def _step_blob_add_only(self):
    """Ajouter une nouvelle arete sans en retirer."""
    if self.N < 2:

```

```

        return False
    u = int(self.rng.integers(self.N))
    v = int(self.rng.integers(self.N))
    if u == v or v in self.adj[u]:
        return False
    u, v = min(u, v), max(u, v)

    self.n_swaps_proposed += 1

    # Calculer ce que cela ferait
    dt_add = len(self.adj[u] & self.adj[v])

    self.adj[u].add(v)
    self.adj[v].add(u)
    self.deg[u] += 1
    self.deg[v] += 1
    new_n_tri = self.n_tri + dt_add
    # Tetraedres crees
    dtetra_add = self._count_tetra_through_edge(u, v)

    new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
    log_alpha = self.N * (new_S - self.S_curr) / TAU
    if log_alpha >= 0:
        accepted = True
    else:
        accepted = self.rng.random() < np.exp(max(log_alpha, -500))

    if accepted:
        self.edge_list.append((u, v))
        self.n_tri = new_n_tri
        self.n_tetra += dtetra_add
        self.S_curr = new_S
        self.n_swaps_accepted += 1
        return True
    else:
        self.adj[u].discard(v)
        self.adj[v].discard(u)
        self.deg[u] -= 1
        self.deg[v] -= 1
        return False

def _step_blob_swap(self, current_step):
    """Swap classique : retirer 1 arete, ajouter 1 autre."""
    idx = int(self.rng.integers(self.E))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))

```

```

if u_add == v_add or v_add in self.adj[u_add]:
    return False
u_add, v_add = min(u_add, v_add), max(u_add, v_add)
if (u_add, v_add) == (u_rm, v_rm):
    return False

self.n_swaps_proposed += 1

dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
dtetra_rm = self._count_tetra_through_edge(u_rm, v_rm)

self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1

dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
dtetra_add = self._count_tetra_through_edge(u_add, v_add)

new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

new_S = np.log(1 + new_n_tri) - BETA * self._compute_D()
log_alpha = self.N * (new_S - self.S_curr) / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))

if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.n_tetra += (dtetra_add - dtetra_rm)
    self.S_curr = new_S

```

```

        self.n_swaps_accepted += 1

    # Detection des premieres apparitions
    if self.first_edge_step is None and self.E > 0:
        self.first_edge_step = current_step
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

def step(self, current_step):
    """Un pas global : decide mitose ou Blob, puis execute."""
    # 1. Mitose ?
    if self.rng.random() < P_MITOSE:
        self.mitose(current_step)
        # Premiere apparition d arete impossible apres mitose seule
        return 'mitose'

    # 2. Pas Blob standard
    accepted = self.step_blob(current_step)

    # Detection premiere arete (au cas ou step_blob_add_only la fait)
    if self.first_edge_step is None and self.E > 0:
        self.first_edge_step = current_step
    if self.first_triangle_step is None and self.n_tri > 0:
        self.first_triangle_step = current_step
    if self.first_tetrahedron_step is None and self.n_tetra > 0:
        self.first_tetrahedron_step = current_step

    return 'blob_accepted' if accepted else 'blob_rejected'

# =====
# RUN PRINCIPAL
# =====
def run_blob_v2(seed, total_steps, label='V2'):

```

```

t_start = time.time()
print('=' * 75)
print('  BLOB 2.0 v1 - Run seed=' + str(seed))
print('  Steps total : ' + str(total_steps))
print('  P_mitose : ' + str(P_MITOSE) + ' par pas')
print('=' * 75)

sim = BlobUniverse(seed=seed)
print('  Init : 1 simplexe a la position centrale, aucune connexion')
print('  Demarrage de l univers...')

# Echantillonnage trajectoire
n_samples = 200
sample_steps = sorted(set([
    max(1, int(np.exp(np.log(total_steps) * i / n_samples)))
    for i in range(1, n_samples + 1)
]))
next_idx = 0
trajectory = []

save_every = 20
save_path = OUTPUT_DIR + '/V2_seed' + str(seed) + '.json'

counters = {'mitose': 0, 'blob_accepted': 0, 'blob_rejected': 0}

for step in range(total_steps):
    action = sim.step(step + 1)
    counters[action] = counters.get(action, 0) + 1

    if next_idx < len(sample_steps) and (step + 1) >= sample_steps[next_idx]:
        traj_point = {
            'step': step + 1,
            'N': sim.N,
            'E': sim.E,
            'n_tri': sim.n_tri,
            'n_tetra': sim.n_tetra,
            'D': sim._compute_D(),
            'S': sim.S_curr,
            'n_mitoses': sim.n_mitoses,
            'time_elapsed': time.time() - t_start,
            'first_edge_step': sim.first_edge_step,
            'first_triangle_step': sim.first_triangle_step,
            'first_tetrahedron_step': sim.first_tetrahedron_step,
        }
        trajectory.append(traj_point)
        next_idx += 1

```

```

if next_idx % save_every == 0:
    # Sauvegarde incrementale
    partial = {
        'seed': seed,
        'partial': True,
        'total_steps_planned': total_steps,
        'steps_done': step + 1,
        'trajectory': trajectory,
        'first_edge_step': sim.first_edge_step,
        'first_triangle_step': sim.first_triangle_step,
        'first_tetrahedron_step': sim.first_tetrahedron_step,
        'final_state': {
            'N': sim.N, 'E': sim.E,
            'n_tri': sim.n_tri, 'n_tetra': sim.n_tetra,
            'n_mitoses': sim.n_mitoses,
            'D': sim._compute_D(), 'S': sim.S_curr
        }
    }
    try:
        with open(save_path, 'w') as f:
            json.dump(partial, f)
    except Exception as e:
        print(' WARN save: ' + str(e))

    # Print progression
    msg = ' step ' + str(step+1) + '/' + str(total_steps)
    msg += ' N=' + str(sim.N)
    msg += ' E=' + str(sim.E)
    msg += ' n_tri=' + str(sim.n_tri)
    msg += ' n_tetra=' + str(sim.n_tetra)
    msg += ' D=' + '{:.4f}'.format(sim._compute_D())
    msg += ' t=' + '{:.0f}'.format(time.time() - t_start) + 's'
    print(msg, flush=True)

    # Annonces de premieres apparitions
    if sim.first_edge_step == step + 1:
        print(' --> PREMIERE ARETE a step=' + str(step+1)
              + ' (N=' + str(sim.N) + ')', flush=True)
    if sim.first_triangle_step == step + 1:
        print(' --> PREMIER TRIANGLE a step=' + str(step+1)
              + ' (N=' + str(sim.N) + ')', flush=True)
    if sim.first_tetrahedron_step == step + 1:
        print(' --> PREMIER TETRAEDRE a step=' + str(step+1)
              + ' (N=' + str(sim.N) + ', n_tri=' + str(sim.n_tri) + ' '
              + ' *** SEUIL THERMODYNAMIQUE ATTEINT ***', flush=True)

total_time = time.time() - t_start

```

```

# Verification finale comptage tetraedres (recount exhaustif)
n_tetra_recount = 0
seen = set()
for u, v in sim.edge_list:
    common = sim.adj[u] & sim.adj[v]
    if len(common) < 2:
        continue
    common_list = sorted(common)
    for i, w in enumerate(common_list):
        for x in common_list[i+1:]:
            if x in sim.adj[w]:
                key = tuple(sorted([u, v, w, x]))
                if key not in seen:
                    seen.add(key)
                    n_tetra_recount += 1

coherence = (sim.n_tetra == n_tetra_recount)

print('')
print('=' * 75)
print('  RUN TERMINE seed=' + str(seed))
print('=' * 75)
print('  Temps : ' + '{:.1f}'.format(total_time/60) + ' min')
print('  Etat final :')
print('    N=' + str(sim.N))
print('    E=' + str(sim.E))
print('    n_tri=' + str(sim.n_tri))
print('    n_tetra=' + str(sim.n_tetra) + ' (recount=' + str(n_tetra_recount)
    + ', coherence=' + ('OK' if coherence else 'KO') + ')')
print('    D=' + '{:.4f}'.format(sim._compute_D()))
print('    S=' + '{:.4f}'.format(sim.S_curr))
print('    n_mitoses=' + str(sim.n_mitoses))
print('  Compteurs :')
for k, v in counters.items():
    print('    ' + k + ' : ' + str(v))
print('  Premieres apparitions :')
if sim.first_edge_step:
    print('    Premiere arete : step ' + str(sim.first_edge_step))
else:
    print('    Premiere arete : JAMAIS')
if sim.first_triangle_step:
    print('    Premier triangle : step ' + str(sim.first_triangle_step))
else:
    print('    Premier triangle : JAMAIS')
if sim.first_tetrahedron_step:
    print('    Premier tetraedre : step ' + str(sim.first_tetrahedron_step))

```

```

else:
    print('    Premier tetraedre : JAMAIS')

# Analyse de la transition tetraedrique si elle a eu lieu
if sim.first_tetrahedron_step:
    # Diviser la trajectoire en pre/post tetraedrique
    pre = [t for t in trajectory if t['step'] < sim.first_tetrahedron_step]
    post = [t for t in trajectory if t['step'] >= sim.first_tetrahedron_step]
    if pre and post:
        d_pre = np.mean([t['D'] for t in pre])
        d_post = np.mean([t['D'] for t in post])
        print('')
        print('  TEST HYPOTHESE COSMOGONIQUE (Mirante) :')
        print('    D moyen avant ler tetraedre : ' + '{:.4f}'.format(d_pre))
        print('    D moyen apres ler tetraedre : ' + '{:.4f}'.format(d_post))
        if d_pre > 0:
            ratio = d_post / d_pre
            print('    Ratio post/pre : ' + '{:.2f}x'.format(ratio))
            if ratio > 1.5:
                print('    -> RUPTURE CONFIRMEE : D augmente avec la matiere
            elif ratio > 1.1:
                print('    -> Effet faible mais detectable')
            else:
                print('    -> Pas de rupture nette')

# Sauvegarde finale
final_data = {
    'seed': seed,
    'partial': False,
    'total_steps': total_steps,
    'time_total_sec': total_time,
    'trajectory': trajectory,
    'first_edge_step': sim.first_edge_step,
    'first_triangle_step': sim.first_triangle_step,
    'first_tetrahedron_step': sim.first_tetrahedron_step,
    'counters': counters,
    'final_state': {
        'N': sim.N, 'E': sim.E,
        'n_tri': sim.n_tri,
        'n_tetra': sim.n_tetra,
        'n_tetra_recount': n_tetra_recount,
        'coherence': coherence,
        'n_mitoses': sim.n_mitoses,
        'D': sim._compute_D(), 'S': sim.S_curr
    }
}

try:

```

```

        with open(save_path, 'w') as f:
            json.dump(final_data, f)
        print('')
        print(' Sauvegarde finale : ' + save_path)
    except Exception as e:
        print(' ERREUR sauvegarde finale : ' + str(e))

    del sim
    gc.collect()
    return final_data

# =====
# MAIN
# =====

print('')
print('=' * 75)
print(' BLOB PRINCIPLE 2.0 v1')
print(' Modele cosmogonique avec mitose simplexiale')
print('=' * 75)
print(' Cadre Rudolphe Mirante :')
print(' - Demarrage : 1 simplexe originel')
print(' - Mitose : ' + str(P_MITOSE) + ' / pas (independant de N)')
print(' - Pas d arete de filiation')
print(' - Decalage spatial epsilon = ' + str(EPS_DECAL) + ' x r_Rips')
print(' - Hypothese a tester : seuil tetraedrique = naissance matiere')
print('=' * 75)

# Configuration
TOTAL_STEPS = 5000000 # 5 millions de pas
N_SEEDS = 2

print('')
print(' Configuration :')
print(' Total pas par seed : ' + str(TOTAL_STEPS))
print(' Nombre de seeds : ' + str(N_SEEDS))
print(' N_mitoses attendu par seed : ~' + str(int(TOTAL_STEPS * P_MITOSE)))
print(' Duree estimee par seed : ~1-3 heures')
print(' Total estime : ~2-6 heures')
print('=' * 75)

t_global = time.time()
all_results = []

for seed in range(N_SEEDS):
    print('')
    try:

```

```

        r = run_blob_v2(seed, TOTAL_STEPS)
        all_results.append(r)
    except Exception as e:
        print('  ERREUR run seed=' + str(seed) + ' : ' + str(e))
        import traceback
        traceback.print_exc()

global_time = time.time() - t_global

# =====
# RAPPORT GLOBAL
# =====

print('')
print('=' * 75)
print('  RAPPORT GLOBAL BLOB 2.0 v1')
print('=' * 75)
print('  Temps total : ' + '{:.1f}'.format(global_time/60) + ' min')
print('')

print('  Resultats par seed :')
for r in all_results:
    s = r['seed']
    fs = r['final_state']
    print('    seed=' + str(s) + ' :')
    print('      N final : ' + str(fs['N']))
    print('      E final : ' + str(fs['E']))
    print('      n_tri : ' + str(fs['n_tri']))
    print('      n_tetra : ' + str(fs['n_tetra']) + ' (recount ' + str(fs['n_tetra_
    print('      D : ' + '{:.4f}'.format(fs['D']))
    print('      lere arete : step ' + str(r['first_edge_step']))
    print('      ler triangle : step ' + str(r['first_triangle_step']))
    print('      ler tetraedre : step ' + str(r['first_tetrahedron_step']))

print('')
print('  Fichiers JSON sauvegardes :')
for f in sorted(os.listdir(OUTPUT_DIR)):
    print('      ' + OUTPUT_DIR + '/' + f)

print('')
print('=' * 75)
print('  FIN BLOB 2.0 v1')
print('=' * 75)

```